Chapter 22: Conventions & Operator Overloading

By Irina Galata

Kotlin is known for its conciseness and expressiveness, which allows you to do more while using less code. Support for user-defined operator overloading is one of the features that gives Kotlin, and many other programming languages, this ability. In this chapter, you'll learn how to efficiently manipulate data by overloading operators.

What is operator overloading?

Operator overloading is primarily *syntactic sugar*, and it allows you to use various operators, like those used in mathematical calculations (e.g., +, −, ∗, +=, >=, etc.) with your custom-defined data types. You can create something like this:

```
val fluffy = Kitten("Fluffy")
val snowflake = Kitten("Snowflake")

takeHome(fluffy + snowflake)
```

You have two instances of the `Kitten` class, and you can take them both home using the + operator. Isn't that nice?

Getting started

For this tutorial, imagine that you run a startup IT company and that you want to build an application to manage your employee and department data.

Open the starter project for this chapter. It has shell classes for `Company`, `Department` and `Employee`. To start, add a list of departments to your `Company` class:

```
private val departments: ArrayList<Department> = arrayListOf()
```

Similarly, add a list of employees to your `Department` class:

```
val employees: ArrayList<Employee> = arrayListOf()
```

And update the `Employee` constructor to track the employee company:

```
class Employee(
  val company: Company,
  val name: String,
  var salary: Int
)
```

Now, update the `main()` function in the starter project to add `company` to each of the employees:

```
fun main(args: Array<String>) {
  // your company
  val company = Company("MyOwnCompany")

  // departments
```

```
    val developmentDepartment = Department("Development")
    val qaDepartment = Department("Quality Assurance")
    val hrDepartment = Department("Human Resources")

    // employees
    var Julia = Employee(company, "Julia", 100_000)
    var John = Employee(company, "John", 86_000)
    var Peter = Employee(company, "Peter", 100_000)

    var Sandra = Employee(company, "Sandra", 75_000)
    var Thomas = Employee(company, "Thomas", 73_000)
    var Alice = Employee(company, "Alice", 70_000)

    var Bernadette = Employee(company, "Bernadette", 66_000)
    var Mark = Employee(company, "Mark", 66_000)
}
```

You have a company consisting of three small departments. As you plan to grow your startup, you'll need to think of the most efficient way of the handling all of the company data, such as departments and staff list, and its processes like hiring, raises or dismissals.

Using conventions

The ability to use overloaded operators in Kotlin is an example of what's called a **convention**. In Kotlin, a convention is an agreement in which you declare and use a function in a specific way, and the prototypical example is being able to use the function with an operator.

In this book, you've already used conventions when you marked a function with the `infix` keyword; in this way, *by convention*, you could omit the function parentheses and call the function without the dot symbol.

Unary operator overloading

You're likely familiar with unary operators in programming languages — `+a`, `--a` or `a++`, for example. If you want to use an increment operator like `++` on your custom data type, you need to declare the `inc()` function as a member of your class, either inside the class or as an extension function. Here, you'll create a function to give your employees a raise. Add the following function to your `Employee` class:

```
operator fun inc(): Employee {
    salary += 5000
    println("$name got a raise to $$salary")
```

```
    return this
  }
```

You mark the function with the `operator` keyword, name it `inc()` and make it return `Employee`. Inside the function, you add 5,000 to the employee `salary`, using the += operator on the `Int`, and you return `this` from the function to return the same instance.

You can now execute employee raises by convention:

```
++Julia // now Julia's salary is 105_000
```

This will be compiled to:

```
Julia = Julia.inc();
```

The decrement operator can be used in a similar way. Add this to `Employee`:

```
operator fun dec(): Employee {
  salary -= 5000
  println("$name's salary decreased to $$salary")
  return this
}
```

For example, you can cut down Peter's salary via:

```
--Peter // now Peter's salary is 95_000
```

Take a look at the convention functions for all the various unary operators:

Operator	Necessary function
++a	inc()
--a	dec()
-a	unaryMinus()
+a	unaryPlus()
!a	not()

You can use these operators to give a meaning to incrementing or decrementing a data type, changing the sign of a data type using – or affirming it with +, and negating it using the not operator `!`.

Binary operator overloading

Similarly, you can use binary operators to combine your custom data types with other values in some kind of meaningful way. An example for our `Employee` class is for employee raises and pay cuts of a specified amount.

Update the `Employee` class by adding the following functions, which will use assigment the operators += and –=:

```
operator fun plusAssign(increaseSalary: Int) {
  salary += increaseSalary
  println("$name got a raise to $$salary")
}

operator fun minusAssign(decreaseSalary: Int) {
  salary -= decreaseSalary
  println("$name's salary decreased to $$salary")
}
```

Inside these functions, you add or subtract the amount passed in as an argument to the employee salary. Since the parameter on the functions is an `Int`, you'll need to combine an `Employee` object with an `Int` using the operators.

Now you can manage the salary of your employees using the corresponding operators:

```
Mark += 2500
Alice -= 2000
```

Kotlin's compilier will translate the code above to the following, as expected:

```
Mark.plusAssign(2500);
Alice.minusAssign(2000);
```

In the same way, you can manage your department and employee lists. Add these functions to the `Company` class:

```
operator fun plusAssign(department: Department) {
  departments.add(department)
}

operator fun minusAssign(department: Department) {
  departments.remove(department)
}
```

And these to the `Department` class:

```
operator fun plusAssign(employee: Employee) {
  employees.add(employee)
  println("${employee.name} hired to $name department")
}

operator fun minusAssign(employee: Employee) {
  if (employees.contains(employee)) {
    employees.remove(employee)
    println("${employee.name} fired from $name department")
  }
}
```

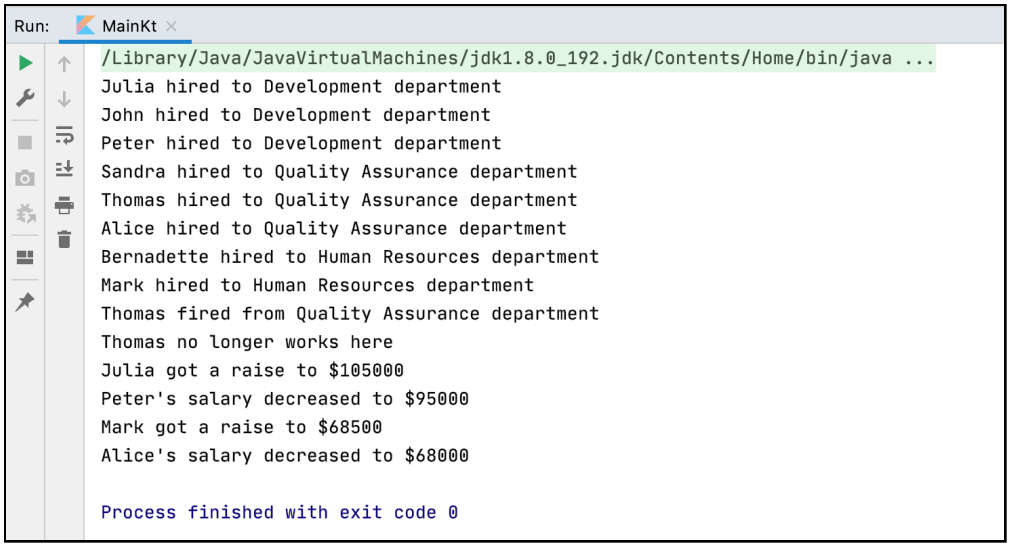
Now, you can concisely manage your company data with the overloaded operators:

```
company += developmentDepartment
company += qaDepartment
company += hrDepartment

developmentDepartment += Julia
developmentDepartment += John
developmentDepartment += Peter

qaDepartment += Sandra
qaDepartment += Thomas
qaDepartment += Alice

hrDepartment += Bernadette
hrDepartment += Mark

qaDepartment -= Thomas
```

These assignments via operator are equivalent to the code below:

```
company.plusAssign(developmentDepartment);
company.plusAssign(qaDepartment);
company.plusAssign(hrDepartment);

developmentDepartment.plusAssign(Julia);
developmentDepartment.plusAssign(John);
developmentDepartment.plusAssign(Peter);

qaDepartment.plusAssign(Sandra);
qaDepartment.plusAssign(Thomas);
qaDepartment.plusAssign(Alice);

hrDepartment.plusAssign(Bernadette);
hrDepartment.plusAssign(Mark);

qaDepartment.minusAssign(Thomas);
```

Build and run the `main()` function and check your console to see the results:



By using the binary assignment operators, you've given yourself an intuitive way to add employees to departments and departments to your startup.

Handling collections

Operator overloading is also quite helpful for when working with collections. For example, if you want to be able to access an employee by its index within a department, declare the following `get()` operator function in the `Department` class:

```
operator fun get(index: Int): Employee? {
  return if (index < employees.size) {
    employees[index]
  } else {
    null
  }
}
```

You first check that the supplied `index` is within the range of employees in the department, and if so, you return that employee from your internal list. Otherwise, you return null. The operator corresponding to this function is the indexing operator:

```
val firstEmployee = qaDepartment[0]
```

Note that this new operator function returns a nullable `Employee`. If you wanted to give that employee a raise, you'd do so as follows:

```
qaDepartment[0]?.plusAssign(1000)
```

Since the return type of the `get()` function is a nullable `Employee?`, you cannot use the `+=` operator directly. You also need to use a safe call operator `?.` to avoid a possible `KotlinNullPointerException` from the code.

If you add the `set()` function to the `Department` class, you'll also be able to set an employee by index:

```
operator fun set(index: Int, employee: Employee) {
  if (index < employees.size) {
    employees[index] = employee
  }
}
```

To update the employee at the second index to Thomas you use the following code:

```
qaDepartment[1] = Thomas
```

To check if an employee works in a given department, you can define the `contains()` operator function in the `Department` class.

Here, you need to confirm if an employee is in the underlying list:

```
operator fun contains(employee: Employee) =
  employees.contains(employee)
```

After adding the function above, you can use `in` and `!in` operators with `Employee` and `Department` objects:

```
if (Thomas !in qaDepartment) {
  println("${Thomas.name} no longer works here")
}
```

Using the indexing and `in` operators makes the logic of your employee and department code much more evident and readable.

Adding ranges

You can also get a list of employees in a given range using the `..` operator. To implement such functionality, first define how to sort the employee list so that you always get the same result from this operator.

Update the `Employee` class to implement the `Comparable` interface:

```
data class Employee(
  val company: Company,
  val name: String,
  var salary: Int
) : Comparable<Employee>
```

Then, override its function `compareTo()`:

```
override operator fun compareTo(other: Employee): Int {
  return when (other) {
    this -> 0
    else -> name.compareTo(other.name)
  }
}
```

By marking the `compareTo()` function with the keyword `operator`, you've ensured that you can use the comparison operators >, <, >=, etc. This function should return `-1`, `0` or `1` if another object is larger, equal to or less than the current one, respectively. You've also marked the class with the keyword `data` to override the `equals()` function so that you can implicitly use it in the`compareTo()` function.

You can now compare employees by their names, which will help you sort them alphabetically. To iterate through the list of employees in a department, update the `Department` class to implement an `Iterable` interface and its function `iterator()`.

Start by updating the class declaration:

```
class Department(
  val name: String = "Department"
) : Iterable<Employee>
```

Then, override `iterator()` to conform to the interface:

```
override fun iterator() = employees.iterator()
```

After that, you can use the following construction on a department:

```
developmentDepartment.forEach {
 // do something
}
```

To access the list of all employees sorted by name in your company, add the following property with custom getter to the `Company` class:

```
val allEmployees: List<Employee>
  get() = arrayListOf<Employee>().apply {
    departments.forEach { addAll(it.employees) }
    sort()
  }
```

Now you can add the `rangeTo()` operator function on the `Employee` class, which corresponds to the `..` operator:

```
operator fun rangeTo(other: Employee): List<Employee> {
  val currentIndex = company.allEmployees.indexOf(this)
  val otherIndex = company.allEmployees.indexOf(other)

  // start index cannot be larger or equal to the end index
  if (currentIndex >= otherIndex) {
    return emptyList()
  }

  // get all elements in a list from currentIndex to otherIndex
  return company.allEmployees.slice(currentIndex..otherIndex)
}
```

Now you can create a range of `Employees`:

```
print((Alice..Mark).joinToString { it.name }) // prints "Alice,
Bernadette, John, Julia, Mark"
```

With the code above, you receive a list of employees from `Alice` to `Mark`, sorted alphabetically. You then join their names to one string and print the result.

You've seen a number of examples of binary and other operators (such as indexing) for custom data types. Below, you can find a list of conventions for the functions corresponding to these and other operators:

Operator	Necessary function	Operator	Necessary function
a + b	plus(b)	a(i, j)	invoke(i, j)
a - b	minus(b)	a..b	rangeTo(b)
a * b	times(b)	a += b	plusAssign(b)
a / b	div(b)	a -= b	minusAssign(b)
a % b	rem(b)	a *= b	timesAssign(b)
a in b	contains(a)	a /= b	divAssign(b)
a[i]	get(i)	a %= b	remAssign(b)
a[i, j]	get(i, j)	a == b	equals(b)
a[i] = b	set(i, b)	a > b	compareTo(b)
a[i, j] = b	set(i, j, b)	a < b	compareTo(b)
a()	invoke()	a >= b	compareTo(b)
a(i)	invoke(i)	a <= b	compareTo(b)

As you review this table, it's important to note that you should be judicious in your use of operator overloading. Any operators that you decide to overload in your data types should be intuitive and recognizable for the given use case, so that they actually make the resulting code not simply more consise but also easier to read and interpret than the more verbose code you would have otherwise used.

Operator overloading and Java

Unlike Kotlin, Java doesn't support user-defined operator overloading. However, the + operator is actually overloaded in standard Java; you not only use it to sum two numbers but also to concatenate strings:

```
String a = "a";
String b = "b";
System.out.print(a + b); // prints "ab"
```

Why doesn't Java allow developers to overload operators themselves?

While overloaded operators can simplify your code, they can also misleading. Since any given operator can have multiple meanings, it can be unclear what's exactly happening in a specific line of code.

As was mentioned at the end of the last section, you should always overload operators attentively, and don't make them behave unexpectedly; for example, the + operator should always be used to "add" two things together in whatever context it is used, and not perform an operation that would correpsond to the equivalent of subtracting, multiplying or dividing.

Delegated properties as conventions

In Chapter 13: "Properties," you were introduced to various types of **delegated propeties**. You can delegate the initialization of a property to another object by using conventions for the `getValue()` and `setValue()` functions in a delegate class:

```
class NameDelegate {
  operator fun getValue(
    thisRef: Any?,
    property: KProperty<*>
  ): String {
    // return existing value
  }
  operator fun setValue(
    thisRef: Any?,
    property: KProperty<*>,
    value: String
  ) {
    // set received value
  }
}
```

In conjunction with the above, you use the construction below to delegate the `name` property to a `NameDelegate` object:

```
var name: String by NameDelegate()
```

In this way, all calls to get or set the `name` property will be delegated to the `getValue()` and `setValue()` functions in `NameDelegate`. This is useful for consolidating complex logic or operations into the delegate class.

Challenges

1. Modify the`Employee` class so that you can add several employees to a department simultaneously using the + operator:

```
developmentDepartment.hire(Julia + John + Peter)
qaDepartment.hire(Sandra + Thomas + Alice)
hrDepartment.hire(Bernadette + Mark)
```

   You'll also need to add a `hire()` function to `Department` that takes a list of employees `List<Employee>` as a parameter.

2. Using the Kotlin Bytecode Viewer in IntelliJ IDEA (**Tools ▸ Kotlin ▸ Show Kotlin Bytecode ▸ Decompile**), review exactly what happens when the code above gets executed.

Key points

- To use **overloaded operators**, it's necessary to follow the specific **conventions** for the operator.
- Conventions manage multiple features in Kotlin, such as operator overloading, infix functions and delegated properties.
- Operators should always behave predictably; don't overload operators in a way that makes their behavior unclear for other developers who might use or read your code.

Where to go from here?

In this chapter, you learned how to add custom behaviors to different operators. Now, you're ready to use them in a real project. Try to replace the routine and repetitive code in your own projects with overloaded operators to make your code more elegant and concise. But don't forget about **predictability** and **clarity**!

In the next chapter, you'll see how to include long-running operations in your code that don't block the rest of your code from running but that still allow you to write your code in a sequential fashion, using **Kotlin Coroutines**.